

Database Notes: Schemas, Keys, and Schema Design

Greyson Knippel
CPSC 321 — Gonzaga University
September 13, 2026

Reference notes for an intro database course. Covers relational schema vocabulary, the key hierarchy, how to derive keys from stated constraints, and foreign key placement in schema design.

1 Relation Schema

A **relation schema** has three parts:

1. **Relation name** — e.g. `STUDENT`
2. **Attributes** — $A_1, A_2, \dots, A_n$ (the columns)
3. **Domains** — $\text{dom}(A_i)$, the allowed atomic values for each attribute

Notation: $R(A_1, A_2, \dots, A_n)$

Example

```
STUDENT (Name, SSN, Age, Major)
```

Name	SSN	Age	Major
Ana Ruiz	305-61-2435	19	CS
Ben Cho	422-11-9087	21	Math
Dee Patel	700-53-1122	20	CS

Term	Meaning	Here
Relation name	the table's name	<code>STUDENT</code>
Attribute	a column	<code>Name, SSN, Age, Major</code>
Domain	allowed values for an attribute	$\text{dom}(\text{Age}) = \text{int } 1\text{--}120$
Degree / arity	number of attributes	4
Tuple	one row	<code><Ana Ruiz, 305-61-2435, 19, CS></code>
Cardinality	number of tuples	3
Attribute value	one cell, must be atomic	$t[\text{Age}] = 19$
Relation state $r(R)$	the whole set of tuples right now	these 3 rows

Schema vs. instance: the schema is the definition (intension); the relation state/instance is the data at a moment (extension).

Relation schema vs. relational database schema: the latter is *all* relation schemas plus integrity constraints.

A relation is a **set of tuples** — no duplicates, no row order. Values *within* a tuple are ordered, matched positionally to the attributes.

2 The Key Hierarchy

Definitions

- **Superkey** — any attribute set that uniquely identifies a tuple. Extras allowed.
- **Candidate key** — a superkey that is **minimal**: removing *any* attribute breaks uniqueness.
- **Primary key** — the candidate key you choose as the official identifier. Never NULL.
- **Alternate key** — the candidate keys you didn't choose.
- **Foreign key** — attribute(s) referencing another relation's primary key.
- **Surrogate key** — a meaningless system-generated ID (auto_increment, UUID).

Simple vs. composite — just a count

- **Simple key** = one attribute
- **Composite key** = two or more attributes taken together

Orthogonal to primary/candidate. A composite key can absolutely be the primary key.

The Minimality Test

Remove each attribute **one at a time**. If **EVERY** removal breaks uniqueness, it's a candidate key. If **ANY** removal survives, it's a superkey only.

Breaking is the PASS condition. If it still works after removal, that attribute was dead weight and the set was not minimal.

One-line version: **a candidate key is a superkey you cannot make any smaller.**

Worked Example — EMPLOYEE

EMPLOYEE(EmpID, Email, Phone, Name, Dept)

Constraints: EmpID, Email, Phone each unique. Name, Dept repeat.

Set	Superkey?	Candidate key?	Why
{EmpID}	✓	✓	unique + minimal
{Email}	✓	✓	unique + minimal
{EmpID, Name}	✓	×	remove Name → still works → not minimal
{EmpID, Email}	✓	×	two keys stapled together — remove either, still works
{Name, Dept}	×	×	not unique
all 5 attributes	✓	×	the whole row is always a superkey

Counts: **28 superkeys** ($2^5 = 32$ subsets, minus the 4 subsets of {Name, Dept}), but only **3 candidate keys**.

Gotcha: {EmpID, Email} is the *worst* offender, not the best — it contains two complete keys when one would do. Keyring analogy: two keys that open the same door. A candidate key is a keyring where losing **any** key locks you out.

Worked Example — CAR (composite candidate key)

CAR(LicensePlate, State, VIN, Make, Model, Year)

License plates are unique only *within* a state.

- Candidate keys: {VIN} (simple) and {LicensePlate, State} (composite)
- {LicensePlate} alone is **not** a superkey — the trap

A relation can have a simple and a composite candidate key at the same time.

Worked Example — Derive From Data

R(A, B, C)

A	B	C
a1	b1	c1
a1	b2	c1
a2	b1	c2
a2	b2	c2

- {A}, {B}, {C} — all have duplicates ×

- $\{A, B\}$ — distinct $\rightarrow$ superkey, and minimal $\rightarrow$ **candidate key** ✓
- $\{A, C\}$ — (a1,c1) twice $\times$ (because $A \rightarrow C$, so C adds nothing)
- $\{B, C\}$ — distinct $\rightarrow$ **candidate key** ✓
- $\{A, B, C\}$ — superkey, not minimal

3 superkeys, 2 candidate keys.

Warning: An instance can only DISPROVE a key, never PROVE one. Keys are constraints over all possible states. Uniqueness in one snapshot may be coincidence. Keys come from the rules of the domain, not from staring at data.

The Minimality Trap

ENROLLMENT(StudentID, CourseID, Semester, Grade)
--

All four columns together are unique — but drop Grade and it still works, so it's a **superkey only**. Candidate key is {StudentID, CourseID, Semester}.

Rule of thumb: **if you can delete an attribute from your answer and it still works, your answer is wrong.**

3 Deriving Keys From Stated Constraints

Word problems state constraints in the form:

“at most one [B] for a given [A]”

This gives you a **functional dependency**: $A \rightarrow B$.

The Rule

1. Translate the sentence into an FD.
2. Ask: **does the left side determine EVERY remaining attribute?**
 - Yes $\rightarrow$ the left side is the key.
 - No $\rightarrow$ add the uncovered attributes, then re-run the minimality test.

Three Worked Cases

Case 1 — ALLOWED_PLAN(*vt_id*, *p_id*)

“A vehicle type has at most one plan; a plan serves many vehicle types.”

- FD: $vt_id \rightarrow p_id$
- Left side covers everything else ✓
- PK = {*vt_id*} (simple)
- Both columns are also FKs. Since it's 1:N, this table could be folded into **VEHICLE_TYPE** as an FK column — junction tables earn their keep only for M:N.

Case 2 — SPY(*agent*, *spies_on*, *spies_for*)

“A country has at most one agent that spies on a given country.”

- FD: $\{spies_on, spies_for\} \rightarrow agent$
- Left side covers everything else ✓
- PK = {*spies_on*, *spies_for*} (composite)
- *agent* is not in the key — a row is an *assignment*, not a person. One agent can hold many assignments (multiple targets, multiple employers, double agent).

Case 3 — SPY(*agent*, *spies_on*, *spies_for*), **different constraint**

“An agent spies on at most one country.”

- FD: $agent \rightarrow spies_on$
- Left side does **not** cover *spies_for* $\times \rightarrow$ patch the hole
- PK = {*agent*, *spies_for*}
- Note: *agent* is only *part* of the key but determines *spies_on* $\rightarrow$ **partial dependency** $\rightarrow$ **2NF violation**. Decomposes into AGENT(*agent*, *spies_on*) and ASSIGNMENT(*agent*, *spies_for*).

Same three columns, same table shape — the constraint decides the key.

4 Foreign Keys

EMPLOYEE(<i>EmpID</i> , <i>Name</i> , <i>DeptID</i>)
--FK--> DEPARTMENT(<i>DeptID</i>)

Referential integrity: every FK value must either match an existing PK value in the parent table, or be NULL.

Three Things Students Get Wrong

1. **An FK does NOT have to be unique.** Many employees, one department — DeptID repeats. Uniqueness is required on the *parent* side only.
2. **An FK CAN be NULL** (unless separately declared NOT NULL). A PK never can.
3. **Direction matters.** The FK lives on the **many** side.

What It Prevents

```
INSERT INTO EMPLOYEE VALUES ('E5', 'Kim', 'D9');    -- REJECTED: D9 doesn't exist (orphan)
DELETE FROM DEPARTMENT WHERE DeptID = 'D1';        -- REJECTED: employees still reference D1
```

Referential Actions

Action	On delete of the parent row
RESTRICT / NO ACTION	refuse the delete (default)
CASCADE	delete the children too
SET NULL	keep children, null out their FK
SET DEFAULT	set FK to a declared default

```
FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
ON DELETE SET NULL
ON UPDATE CASCADE
```

Variations

- **Composite FK** — if the parent PK is composite, the FK must be too, all columns together.
- **Self-referencing FK** — EMPLOYEE(EmpID, Name, ManagerID) where ManagerID → EMPLOYEE(EmpID). The top of the chain has ManagerID = NULL, which is exactly why FKs must allow NULL.
- **PK that is also an FK** — normal in junction tables and 1:1 relationships.

5 Schema Design — Where the FK Goes

Relationship	Where the FK goes
1:N	FK column on the many side — no extra table
M:N	New junction table , PK = both FKs combined
1:1	FK on either side, plus a UNIQUE constraint

Cardinality decides this, not table shape. Two two-column tables can look identical and need opposite designs.

M:N Example — Books and Authors

“A book can have multiple authors, and an author can write multiple books.”

```
BOOK(isbn, title, pub_year)
    PK: {isbn}

AUTHOR(author_id, first_name, last_name)
    PK: {author_id}

WROTE(isbn, author_id)
    PK: {isbn, author_id}
    FK: isbn -> BOOK(isbn)
    FK: author_id -> AUTHOR(author_id)
```

BOOK and AUTHOR have no foreign keys. Both FKs live in the junction table.

Why the third table is mandatory: putting `author_id = "A1, A2, A3"` in `BOOK` violates **atomicity (1NF)**; `author1/author2/author3` columns cap the author count and fill with NULLs. The junction table makes each authorship its own row.

```
CREATE TABLE WROTE (
    isbn          CHAR(13),
    author_id     INTEGER,
    PRIMARY KEY (isbn, author_id),
    FOREIGN KEY (isbn) REFERENCES BOOK(isbn) ON DELETE
        CASCADE,
    FOREIGN KEY (author_id) REFERENCES AUTHOR(author_id) ON DELETE
        CASCADE
);
```

Relationship attributes (e.g. `author_position` for cover order) belong in the junction table — they describe the pairing, not either entity, and don’t join the key.

1:N Example — Pets and Owners

“Each pet belongs to exactly one owner.”

```
OWNER(owner_id, name, phone)
    PK: {owner_id}

PET(pet_id, name, species, owner_id)
    PK: {pet_id}
    FK: owner_id -> OWNER(owner_id)    [NOT NULL]
```

Two tables, no junction table. A HAS_PET junction table would *permit* a pet with two owners — **your schema should make illegal states impossible**.

Don't put `pet_id` in OWNER — that caps each owner at one pet.

The “Exactly” vs. “At Most” Distinction

“exactly one” → FK is NOT NULL

“at most one” → FK allows NULL

Graders look for this. It's also the one real argument for keeping a separate table in a 1:N case: if most rows would have no match, a separate table avoids a column full of NULLs.

6 OLTP vs. OLAP

	OLTP	OLAP
Purpose	runs the business	analyzes the business
Workload	many small short transactions	few huge complex read queries
Query touches	a few rows, by key	millions of rows, scanned + aggregated
Read/write	write-heavy, high concurrency	read-mostly, bulk-loaded
Data	current, detailed, updated in place	historical, summarized, non-volatile
Design	normalized (3NF) — cheap writes	denormalized — cheap joins
Optimizes for	ACID, latency, throughput	scan performance

Why they're separate databases: a 40-minute analytical scan shouldn't lock up order processing, and the two workloads want physically opposite designs.

Warehouse Vocabulary

Data warehouse (Inmon's definition — often tested verbatim): *subject-oriented, integrated, time-variant, non-volatile*.

- *Subject-oriented* — organized around subjects, not applications
- *Integrated* — merged from many sources, formats reconciled
- *Time-variant* — every record carries a time dimension; history kept
- *Non-volatile* — loaded and read, not updated or deleted

- **Data mart** — a warehouse scoped to one department
- **ETL** — Extract, Transform, Load (ELT = load raw, transform in the warehouse)
- **Star schema** — central **fact table** (numeric measures + FKs) surrounded by denormalized **dimension tables** (Product, Store, Date, Customer)
- **Snowflake schema** — star schema with normalized dimensions
- **Fact vs. dimension** — facts are measures; dimensions are how you slice them

OLAP Cube Operations

Operation	Meaning
Roll-up	aggregate up a hierarchy (day $\rightarrow$ month $\rightarrow$ year)
Drill-down	the reverse, go finer
Slice	fix one dimension to a single value
Dice	subcube from ranges on several dimensions
Pivot	rotate the axes of the view

Quick Reference Card

Superkey = unique.

Candidate key = unique AND nothing removable.

Primary key = the candidate key you picked.

Composite = 2+ attributes. **Simple** = 1 attribute.

Minimality test: remove each attribute one at a time — *every* removal must break it.

Constraint $\rightarrow$ key: turn the sentence into an FD, check whether the left side covers all remaining attributes, patch if not.

FK placement: 1:N $\rightarrow$ many side. M:N $\rightarrow$ junction table with composite PK.

“Exactly one” $\rightarrow$ NOT NULL. “At most one” $\rightarrow$ NULL allowed.

Instances disprove keys; they never prove them.